

Môn An toàn và bảo mật thông tin.

1. - Tìm hiểu thuật toán mã hoá hiện đại DES, AES

- Mô tả đc thuật toán, quy trình mã hoá/giải mã
- Cài đặt AES trên 1 ngôn ngữ lập trình nào đó

1.1 Tìm hiểu thuật toán hiện đại DES

DES (Data Encryption Standard) là thuật toán mã hóa đối xứng được IBM phát triển dựa trên thuật toán Lucifer. DES được chuẩn hóa và sử dụng rộng rãi trong nhiều hệ thống bảo mật trước đây.

DES mã hóa dữ liệu theo từng khối có kích thước:

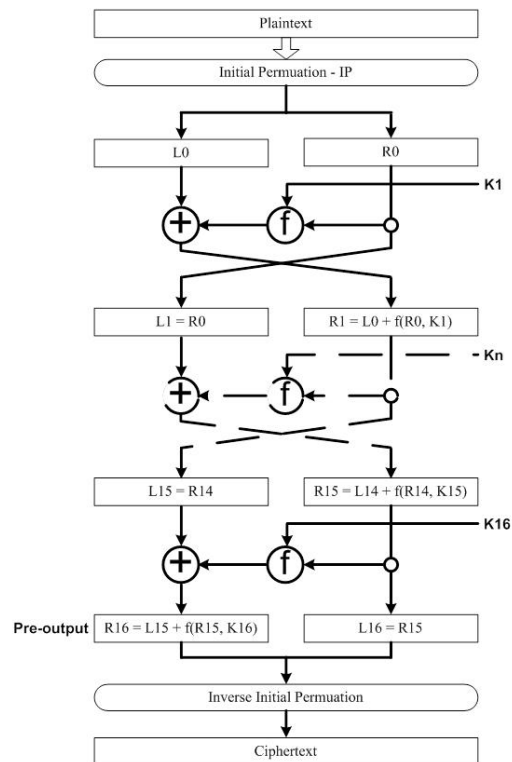
- Kích thước khối: 64 bit
- Khóa đầu vào: 64 bit
- Trong đó có 8 bit dùng cho kiểm tra chẵn lẻ
- Khóa thực sự được sử dụng: 56 bit
- Số vòng mã hóa: 16 vòng

Hiện nay DES không còn được xem là thuật toán an toàn cho các hệ thống mới vì khóa 56 bit quá ngắn và có thể bị tấn công vét cạn.

1.1.1 Lưu đồ thuật toán mã hóa

Thuật toán DES được sử dụng để mã hóa và giải mã các block (khối) dữ liệu 64 bit dựa trên một key (khóa mã) 64 bit. Chú ý, các block được đánh số thứ tự bit từ trái sang phải và bắt đầu từ 1, bit đầu tiên bên trái là bit số 1 và bit cuối cùng bên phải là bit số 64. Quá trình giải mã và mã hóa sử dụng cùng một key nhưng thứ tự phân phối các giá trị các bit key của quá trình giải mã ngược với quá trình mã hóa.

Một block dữ liệu sẽ được hoán vị khởi tạo (Initial Permutation) IP trước khi thực hiện tính toán mã hóa với key. Cuối cùng, kết quả tính toán với key sẽ được hoán vị lần nữa để tạo ra , đây là hoán vị đảo của hoán vị khởi tạo gọi là (Inverse Initial Permutation) IP-1. Việc tính toán dựa trên key được định nghĩa đơn giản trong một hàm f, gọi là hàm mã hóa, và một hàm KS, gọi là hàm phân phối key (key schedule). Hàm KS là hàm tạo ra các khóa vòng (round key) cho các lần lặp mã hóa. Có tất cả 16 khóa vòng từ K1 đến K16.

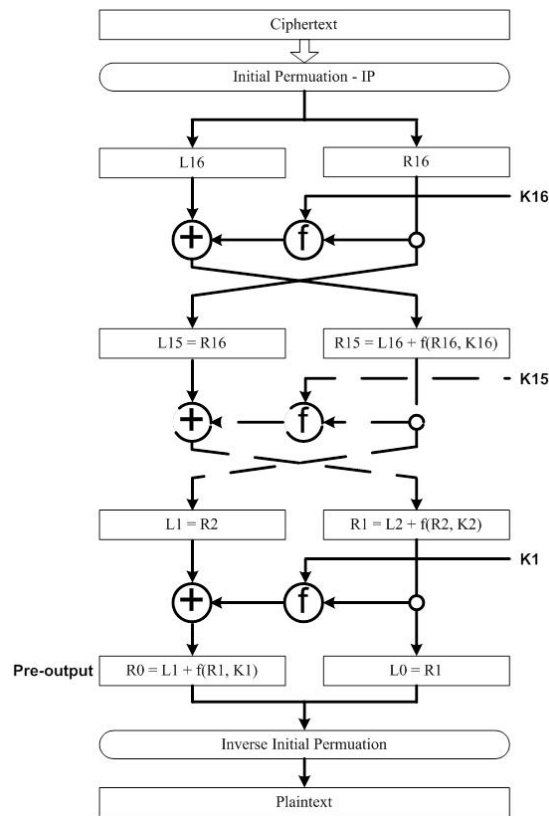


Hình 1.1 Giải thuật mã hóa DES

1.1.2 Thuật toán giải mã dữ liệu DES

Các bước của quá trình giải mã dữ liệu được thực hiện tương tự như quá trình mã hóa dữ liệu. Trong quá trình giải mã có một số thay đổi như sau:

- Đầu vào lúc này là dữ liệu cần giải mã (ciphertext) và đầu ra là kết quả giải mã được (plaintext).
- Khóa vòng sử dụng trong các vòng lặp giải mã có thứ tự ngược với quá trình mã hóa. Nghĩa là, tại vòng lặp giải mã đầu tiên, khóa vòng được sử dụng là K16. Tại vòng lặp giải mã thứ 2, khóa vòng được sử dụng là K15, và tại vòng lặp giải mã cuối cùng thì khóa vòng được sử dụng là K1.



Hình 1.2 Quá trình giải mã dữ liệu DES

1.2 Tìm hiểu thuật toán hiện đại AES

AES (viết tắt của Advanced Encryption Standard) là một thuật toán mã hóa đối xứng phổ biến để bảo vệ dữ liệu trong các hệ thống máy tính và mạng. Được Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST) công bố vào năm 2001, AES ra đời đã thay thế chuẩn mã hóa DES (Data Encryption Standard) trước đây.

Thuật toán AES sử dụng cùng một khóa cho cả việc mã hóa và giải mã dữ liệu, đây là đặc trưng của mã hóa đối xứng. AES làm việc trên các khối dữ liệu có kích thước 128 bit và có thể sử dụng các khóa với độ dài khác nhau (128, 192 hoặc 256 bit) để tăng cường mức độ bảo mật.

1.2.1 Tổng quan

AES là một mã khối, nhưng khác với các mã khối khác được biết đến trước đây (DES, IDEA,...), dữ liệu trong AES không được biểu diễn dưới dạng một mảng các byte hay các bit mà được biểu diễn dưới dạng một ma trận $4 \times Nb$ và được gọi là mảng trạng thái (state). Trong đó, đối với AES, Nb luôn có giá trị bằng 4. Trong khi thuật toán Rijndael hỗ trợ ba giá trị của Nb là 4, 6, 8 tương ứng với kích thước khối 128, 192 và 256 bit. Dữ liệu đầu vào được đọc vào ma trận state theo từng cột, theo thứ tự từ trên xuống dưới, từ trái qua phải. Dữ liệu đầu ra được đọc từ ma trận cũng theo quy tắc trên.

$$S = \begin{pmatrix} s_{00} & s_{01} & s_{02} & s_{03} \\ s_{10} & s_{11} & s_{12} & s_{13} \\ s_{20} & s_{21} & s_{22} & s_{23} \\ s_{30} & s_{31} & s_{32} & s_{33} \end{pmatrix}$$

Khóa vòng trong AES cũng được biểu diễn hoàn toàn tương tự như cách biểu diễn dữ liệu. Tuy nhiên, tùy vào kích thước khóa mà số cột của ma trận khóa vòng N_k sẽ khác nhau. Cụ thể, N_k nhận các giá trị 4, 6, 8 tương ứng với các kích thước khóa là 128, 192 và 256 bit. Đây chính là điểm mạnh của thuật toán AES trong vấn đề mở rộng khóa.

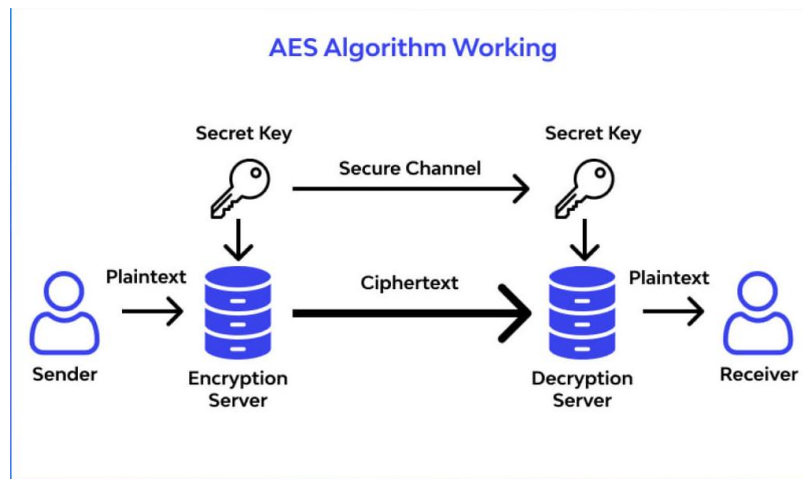
$$k = \begin{pmatrix} k_{00} & k_{01} & \dots & k_{0,N_k} \\ k_{10} & k_{11} & \dots & k_{1,N_k} \\ k_{20} & k_{21} & \dots & k_{2,N_k} \\ k_{30} & k_{31} & \dots & k_{3,N_k} \end{pmatrix}$$

Số vòng lặp ký hiệu là N_r , phụ thuộc vào hai đại lượng N_b và N_k . Vì N_b trong AES có giá trị cố định nên N_r chỉ phụ thuộc vào N_k . Giá trị của N_r tương ứng với ba giá trị của N_k là $N_r = 10, 12, 14$. Cụ thể, giá trị N_r được xác định bởi

N_k	4	6	8
N_r	10	12	14

1.2.2 Thuật toán của AES

AES bao gồm ba loại mật mã khối là AES-128, AES-192 và AES-256, tương ứng với các chiều dài khóa là 128 bit, 192 bit và 256 bit. Mỗi loại khóa có số vòng lặp khác nhau tương ứng với 10 vòng cho khóa 128 bit, 12 vòng cho khóa 192 bit và 14 vòng cho khóa 256 bit. Trong mỗi vòng lặp, ba bước thay thế, biến đổi và hòa trộn khối văn bản gốc sẽ được thực hiện để chuyển đổi nó thành văn bản mã hóa.



Chính phủ phân loại thông tin thành ba cấp độ: bảo mật, bí mật và tối mật. Các khóa có độ dài 128, 192 và 256 bit đều được sử dụng cho thông tin bảo mật và bí mật. Riêng đối với thông tin tối mật, để đảm bảo hệ thống dữ liệu an toàn tuyệt đối, chỉ có các khóa 192 hoặc 256 bit mới được áp dụng. Mã hóa sẽ sử dụng một khóa bí mật để mã hóa và giải mã dữ liệu, đòi hỏi cả người gửi lẫn người nhận đều phải biết và sử dụng khóa này.

1.3 Cài đặt AES trên 1 ngôn ngữ lập trình python

Quy trình mã hóa trong chương trình

Khi người dùng nhập bản rõ và khóa AES rồi nhấn nút Mã hóa, chương trình thực hiện các bước sau:

Bước 1: Nhận dữ liệu từ giao diện web.

Bước 2: Kiểm tra độ dài khóa. Khóa phải có 16, 24 hoặc 32 byte tương ứng với AES-128, AES-192 hoặc AES-256.

Bước 3: Chuyển dữ liệu văn bản sang dạng byte bằng mã hóa UTF-8.

Bước 4: Thực hiện Padding để dữ liệu có kích thước phù hợp với kích thước khối của AES.

Bước 5: Khởi tạo thuật toán AES với khóa được cung cấp.

Bước 6: Thực hiện mã hóa dữ liệu để tạo Ciphertext.

Bước 7: Chuyển Ciphertext sang Base64 để thuận tiện hiển thị và truyền qua giao diện web.

Bước 8: Hiển thị kết quả trên trình duyệt.

2. Tìm hiểu về thuật toán mã hoá bất đối xứng RSA nguyên lý sinh cặp khoá bí mật, công khai

2.1 Thuật toán mã hoá bất đối xứng RSA

RSA là một hệ mã hóa bất đối xứng (asymmetric cryptography) sử dụng hai khóa khác nhau để mã hóa và giải mã. Public key (khóa công khai) được chia sẻ với bất kỳ ai và dùng để mã hóa thông tin, trong khi private key (khóa bí mật) được giữ kín và chỉ người sở hữu nó mới có thể giải mã thông tin.

Bằng cách sử dụng public key, bất kỳ ai cũng có thể gửi thông tin một cách an toàn, nhưng chỉ người giữ private key mới có thể đọc được thông điệp. Điều này mang lại mức độ bảo mật cao vì dù thông tin có bị chặn, chỉ người có private key mới có thể giải mã.

Dù trên lý thuyết RSA có độ bảo mật rất cao, trong thực tế không có phương pháp nào đảm bảo an toàn tuyệt đối. Với sự phát triển của công nghệ như AI, máy tính lượng tử và siêu máy tính,... các hệ thống mã hóa hiện tại có thể trở nên dễ bị phá vỡ hơn. Hiện tại, các thuật toán khóa như RSA vẫn bảo vệ được thông tin trước các kỹ thuật tấn công thông thường, đặc biệt khi sử dụng máy tính cá nhân, nhưng trong tương lai có thể bị đe dọa bởi các công nghệ tiên tiến hơn.

2.2 Nguyên lý sinh cặp khoá bí mật, công khai

Hoạt động của RSA dựa trên 4 bước chính: sinh khóa, chia sẻ key, mã hóa và giải mã.

*Quá trình sinh hóa

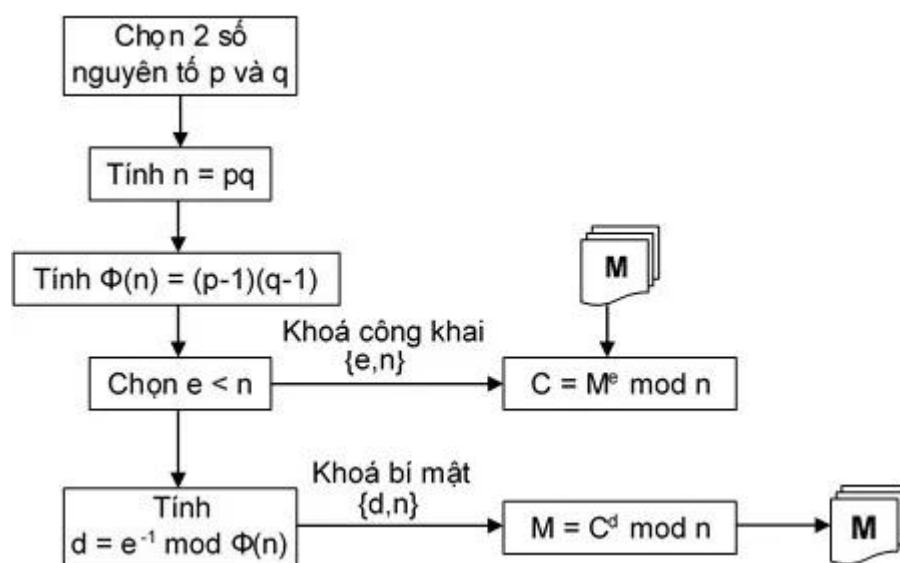
- Việc tạo khóa trong RSA dựa trên việc tìm ra bộ ba số tự nhiên: e, d, và n, với yêu cầu rằng khi mã hóa và giải mã thông điệp m, công thức sau được thỏa mãn:

$$m^e \bmod n = m^d \bmod n$$

Một điểm quan trọng là private key d phải được bảo mật tuyệt đối. Ngay cả khi ai đó biết được e, n, hay thông điệp m, họ cũng không thể tính được d. Cụ thể, quá trình sinh khóa trong RSA gồm các bước như sau:

- Chọn hai số nguyên tố lớn p và q: Đây là hai số bí mật mà chỉ người tạo khóa mới biết.
- Tính giá trị $n = p * q$: Giá trị này sẽ được dùng làm modulus cho cả public key và private key.

- Tính phi hàm Carmichael $\lambda(n)$: Đây là một số giả nguyên tố được tính bằng cách lấy bội chung nhỏ nhất (BCNN) của $\lambda(p)$ và $\lambda(q)$, với $\lambda(p) = p - 1$ và $\lambda(q) = q - 1$. Giá trị $\lambda(n)$ sẽ được giữ bí mật.
- Chọn số tự nhiên e : Chọn một số e trong khoảng $(1, \lambda(n))$ sao cho $\text{UCLN}(e, \lambda(n)) = 1$, nghĩa là e và $\lambda(n)$ nguyên tố cùng nhau. Số e sẽ được dùng để mã hóa thông điệp.
- Tính số d : Tìm số d sao cho $d * e \equiv 1 \pmod{\lambda(n)}$, hay nói cách khác, d là nghịch đảo modulo của e theo $\lambda(n)$. Số d này sẽ được dùng để giải mã thông điệp.
- Public key: Là bộ số (n, e) , và có thể chia sẻ công khai.
- Private key: Là bộ số (n, d) , cần được giữ bí mật.



Hình 2.1 Quá trình sinh hóa

Để đảm bảo an toàn, cần bảo mật các số nguyên tố p và q , vì nếu chúng bị lộ, quá trình sinh khóa có thể bị phá vỡ.

* Mã hóa và giải mã trong RSA Encryption

Trong quá trình mã hóa và giải mã bằng RSA, chúng ta sử dụng public key (n, e) để mã hóa và private key (n, d) để giải mã. Dưới đây là các bước chi tiết:

- Mã hóa: Khi có một bản rõ (thông điệp) M , ta cần chuyển nó thành một số tự nhiên m sao cho $0 < m < n$ và m nguyên tố cùng nhau với n . Điều này có thể thực hiện dễ dàng bằng cách sử dụng các kỹ thuật padding (thêm dữ liệu bổ sung). Sau đó, ta tiến hành mã hóa số m thành c (bản mã) bằng công thức:

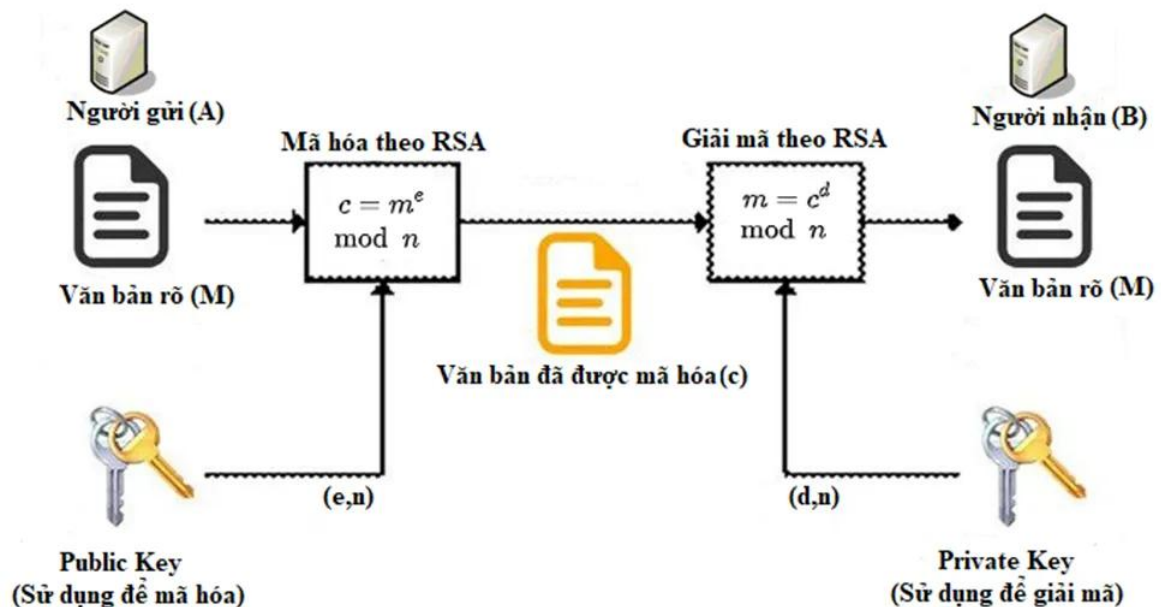
$$c = m^e \pmod{n}$$

- Giá trị c sau đó sẽ được gửi tới người nhận.

- Giải mã: Người nhận sử dụng private key (n, d) để giải mã bản mã c nhằm lấy lại số m bằng công thức:

$$m = c^d \bmod n$$

- Sau khi có m, ta có thể khôi phục lại bản tin ban đầu M bằng cách đảo ngược quá trình padding.



Hình 2.2 Mã hóa và giải mã trong RSA

* Cơ chế hoạt động chữ ký số

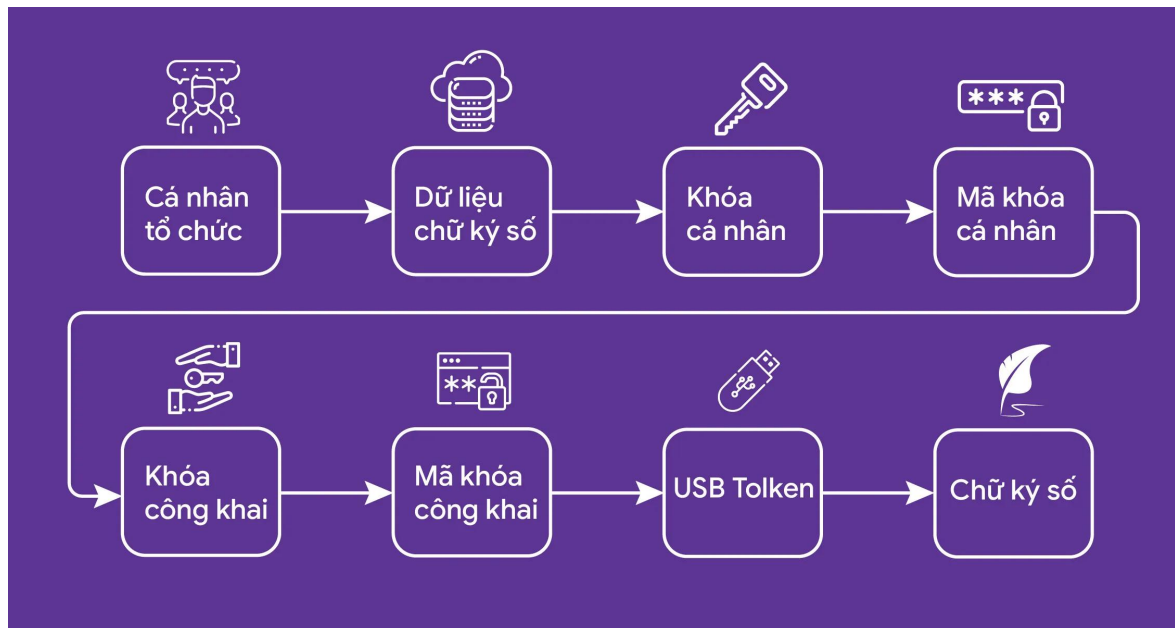
Chữ ký số dựa trên hệ mã hóa RSA hoạt động tương tự như quá trình mã hóa và giải mã thông tin. Tuy nhiên, vai trò của public key và private key trong chữ ký số có sự thay đổi:

- Tạo chữ ký: Người gửi sử dụng private key của mình để tạo ra chữ ký số.
- Xác thực chữ ký: Người nhận dùng public key của người gửi để xác thực tính hợp lệ của chữ ký.
- Cách tạo và xác thực chữ ký số

Vì việc mã hóa toàn bộ bản tin có thể tốn thời gian và không hiệu quả, thay vì mã hóa cả bản tin, chỉ giá trị hash của bản tin được mã hóa. Phương pháp này có nhiều ưu điểm:

- Tính một chiều của hàm hash: Hàm hash là một hàm một chiều, do đó, ngay cả khi biết giá trị hash, cũng không thể khôi phục lại bản tin gốc.
- Độ dài cố định: Giá trị hash có độ dài cố định và nhỏ, giúp giảm dung lượng của chữ ký số.

- Kiểm tra tính toàn vẹn: Hash còn giúp kiểm tra xem bản tin có bị thay đổi trong quá trình truyền tải hay không. Nếu giá trị hash của bản tin không trùng khớp, nghĩa là dữ liệu đã bị thay đổi.



Hình 2.3 Cách tạo và xác thực chữ ký số

3. - Trình bày các mô hình hình áp dụng thuật toán RSA

- Xác thực người gửi, xác thực người nhận, cả 2
- So sánh thời gian mã hoá/giải mã của RSA với AES.
- Đưa ra các cách dùng kết hợp sức mạnh của RSA và AES.

* Trình bày các mô hình hình áp dụng thuật toán RSA, xác thực người gửi, xác thực người nhận, cả 2

Mô hình thứ nhất là RSA dùng để bảo mật thông tin cho người nhận. Khi người gửi A muốn gửi một thông điệp bí mật cho người nhận B, A sử dụng khóa công khai của B để mã hóa thông điệp. Sau khi được mã hóa, thông điệp trở thành bản mã và có thể được truyền qua mạng mà không cần lo ngại người khác đọc được nội dung. Khi nhận được bản mã, B sử dụng khóa bí mật của mình để giải mã và lấy lại thông điệp ban đầu. Do chỉ B sở hữu khóa bí mật tương ứng nên mô hình này đảm bảo tính bí mật của thông tin. Có thể biểu diễn quá trình bằng công thức $C=E(K_B^+,M)$ trong đó M là thông điệp, K_B^+ là khóa công khai của B và C là bản mã. B giải mã bằng K_B^- theo công thức $M=D(K_B^-,C)$.

Mô hình thứ hai là RSA dùng để xác thực người gửi thông qua chữ ký số. Trong trường hợp này, người gửi A sử dụng khóa bí mật của mình để tạo chữ ký

cho thông điệp. Thông thường, A sẽ thực hiện hàm băm đối với thông điệp để tạo ra một giá trị đại diện, sau đó dùng khóa bí mật để tạo chữ ký số từ giá trị băm đó. A gửi thông điệp cùng chữ ký cho B. Khi nhận được, B sử dụng khóa công khai của A để kiểm tra chữ ký và đồng thời tính lại giá trị băm của thông điệp. Nếu hai giá trị trùng nhau và chữ ký hợp lệ, B có thể xác nhận rằng thông điệp được tạo bởi người sở hữu khóa bí mật của A và nội dung không bị thay đổi trong quá trình truyền. Mô hình này cung cấp khả năng xác thực người gửi và đảm bảo tính toàn vẹn của dữ liệu.

Mô hình thứ ba là kết hợp RSA để mã hóa và xác thực trong cùng một quá trình. Trong mô hình này, người gửi A trước tiên tạo chữ ký số bằng khóa bí mật của mình để xác thực nguồn gốc thông điệp. Sau đó, thông điệp và chữ ký có thể được mã hóa bằng khóa công khai của người nhận B. Khi B nhận được dữ liệu, B sử dụng khóa bí mật của mình để giải mã, sau đó sử dụng khóa công khai của A để kiểm tra chữ ký. Nhờ đó, hệ thống có thể đồng thời đảm bảo tính bí mật của nội dung, xác thực người gửi và kiểm tra tính toàn vẹn của thông điệp.

Mô hình thứ tư là kết hợp RSA với AES để tạo ra hệ thống mã hóa lai. Trong thực tế, RSA không phù hợp để mã hóa trực tiếp những dữ liệu có kích thước lớn vì tốc độ xử lý thấp hơn các thuật toán mã hóa đối xứng. Vì vậy, hệ thống có thể sử dụng AES để mã hóa dữ liệu và RSA để bảo vệ khóa AES. Người gửi tạo một khóa AES ngẫu nhiên và sử dụng khóa này để mã hóa dữ liệu. Sau đó, khóa AES được mã hóa bằng khóa công khai RSA của người nhận và gửi cùng bản mã đến người nhận. Người nhận sử dụng khóa bí mật RSA của mình để lấy lại khóa AES, sau đó sử dụng khóa AES để giải mã dữ liệu. Cách kết hợp này tận dụng được tốc độ của AES và khả năng bảo vệ khóa của RSA, phù hợp với các hệ thống truyền và lưu trữ dữ liệu lớn.

Như vậy, RSA có thể được áp dụng theo nhiều mô hình khác nhau tùy vào yêu cầu bảo mật của hệ thống. RSA có thể được sử dụng để mã hóa thông tin nhằm bảo vệ người nhận, sử dụng khóa bí mật để tạo chữ ký số nhằm xác thực người gửi, hoặc kết hợp cả hai để đảm bảo đồng thời tính bí mật, tính toàn vẹn và xác thực. Trong các hệ thống thực tế, RSA thường được kết hợp với AES, trong đó AES chịu trách nhiệm mã hóa dữ liệu lớn còn RSA đảm nhiệm việc bảo vệ khóa AES. Đây là mô hình kết hợp giúp hệ thống đạt được hiệu quả và mức độ bảo mật cao hơn.

* So sánh RSA và AES

Đặc điểm	RSA (Bất đối xứng)	AES (Đối xứng)
----------	--------------------	----------------

Loại mã hóa	Sử dụng cặp khóa công khai-riêng	Sử dụng một khóa chung
Độ dài khóa	2048, 3072, 4096 bit	128, 192, 256 bit
Tốc độ	Chậm, chi phí tính toán cao	Nhanh, hiệu quả với dữ liệu lớn
Ứng dụng chính	Chữ ký số, SSL/TLS	Lưu trữ dữ liệu, bảo mật VPN
Ưu điểm	Bảo mật cao cho mã hóa công khai và chữ ký số.	Nhanh và hiệu quả với lượng dữ liệu lớn.
	Phù hợp với trao đổi khóa an toàn.	Yêu cầu ít tài nguyên tính toán.
Nhược điểm	Chậm, tốn nhiều tài nguyên.	Dễ gặp vấn đề nếu quản lý khóa không tốt.
	Không phù hợp để mã hóa dữ liệu lớn.	Khóa dùng chung phải được bảo vệ chặt chẽ.

* Đưa ra các cách dùng kết hợp sức mạnh của RSA và AES.

Các cách kết hợp sức mạnh của RSA và AES

RSA và AES là hai thuật toán có đặc điểm khác nhau nhưng có thể kết hợp để tận dụng ưu điểm của cả mã hóa bất đối xứng và mã hóa đối xứng. AES có tốc độ mã hóa và giải mã nhanh, phù hợp với dữ liệu có kích thước lớn, trong khi RSA có ưu điểm trong việc bảo vệ và trao đổi khóa. Vì vậy, trong thực tế người ta thường sử dụng RSA để bảo vệ khóa AES thay vì dùng RSA để mã hóa toàn bộ dữ liệu.

Cách thứ nhất là sử dụng AES để mã hóa dữ liệu và RSA để mã hóa khóa AES. Người gửi trước tiên tạo một khóa AES ngẫu nhiên, sau đó sử dụng khóa này để mã hóa dữ liệu cần truyền. Vì khóa AES có kích thước nhỏ nên người gửi tiếp tục sử dụng khóa công khai RSA của người nhận để mã hóa khóa AES. Sau đó, người gửi gửi cho người nhận cả bản mã dữ liệu và khóa AES đã được mã hóa bằng RSA. Người nhận sử dụng khóa bí mật RSA của mình để giải mã và lấy lại khóa

AES, sau đó dùng khóa AES để giải mã dữ liệu ban đầu. Cách này tận dụng tốc độ xử lý nhanh của AES và khả năng bảo vệ khóa của RSA.

Cách thứ hai là sử dụng AES để mã hóa dữ liệu và RSA để tạo chữ ký số. Trong mô hình này, dữ liệu trước tiên được mã hóa bằng AES nhằm đảm bảo tính bí mật. Đồng thời, người gửi có thể tạo giá trị băm của dữ liệu hoặc bản mã rồi sử dụng khóa bí mật RSA để tạo chữ ký số. Người nhận sau khi nhận dữ liệu sẽ dùng khóa AES để giải mã và sử dụng khóa công khai RSA của người gửi để kiểm tra chữ ký. Nếu chữ ký hợp lệ, người nhận có thể xác nhận nguồn gốc và kiểm tra tính toàn vẹn của dữ liệu. Như vậy, AES đảm bảo bí mật, còn RSA đảm bảo xác thực và toàn vẹn.

Cách thứ ba là kết hợp AES, RSA và chữ ký số trong một hệ thống bảo mật hoàn chỉnh. Người gửi sử dụng AES để mã hóa dữ liệu, sử dụng RSA với khóa công khai của người nhận để bảo vệ khóa AES, đồng thời sử dụng khóa bí mật RSA của mình để tạo chữ ký số. Khi nhận được dữ liệu, người nhận sử dụng khóa bí mật RSA để lấy khóa AES, dùng AES để giải mã dữ liệu và dùng khóa công khai RSA của người gửi để kiểm tra chữ ký. Mô hình này có thể đồng thời đảm bảo tính bí mật, xác thực người gửi và tính toàn vẹn của dữ liệu.

Tài liệu tham khảo

- [1] <https://ezyplatform.com/blog/thuat-toan-rsa-va-aes->
- [2] <https://eca.com.vn/tin-tuc/thuat-toan-rsa-la-gi>
- [3] <https://vietnix.vn/rsa/>
- [4] <https://viblo.asia/p/he-ma-hoa-rsa-va-chu-ky-so-6J3ZgkgMZmB>
- [5] <https://viblo.asia/p/he-ma-hoa-rsa-va-chu-ky-so-6J3ZgkgMZmB>
- [6]